

LibPro — Use Case & Architecture

A UML-based overview of the Library Management System: actors, use cases, sequence flows, class structure, and MVC architecture.



Use Case Diagram

Reader

Search books, request loan extension, update contact info.

Librarian

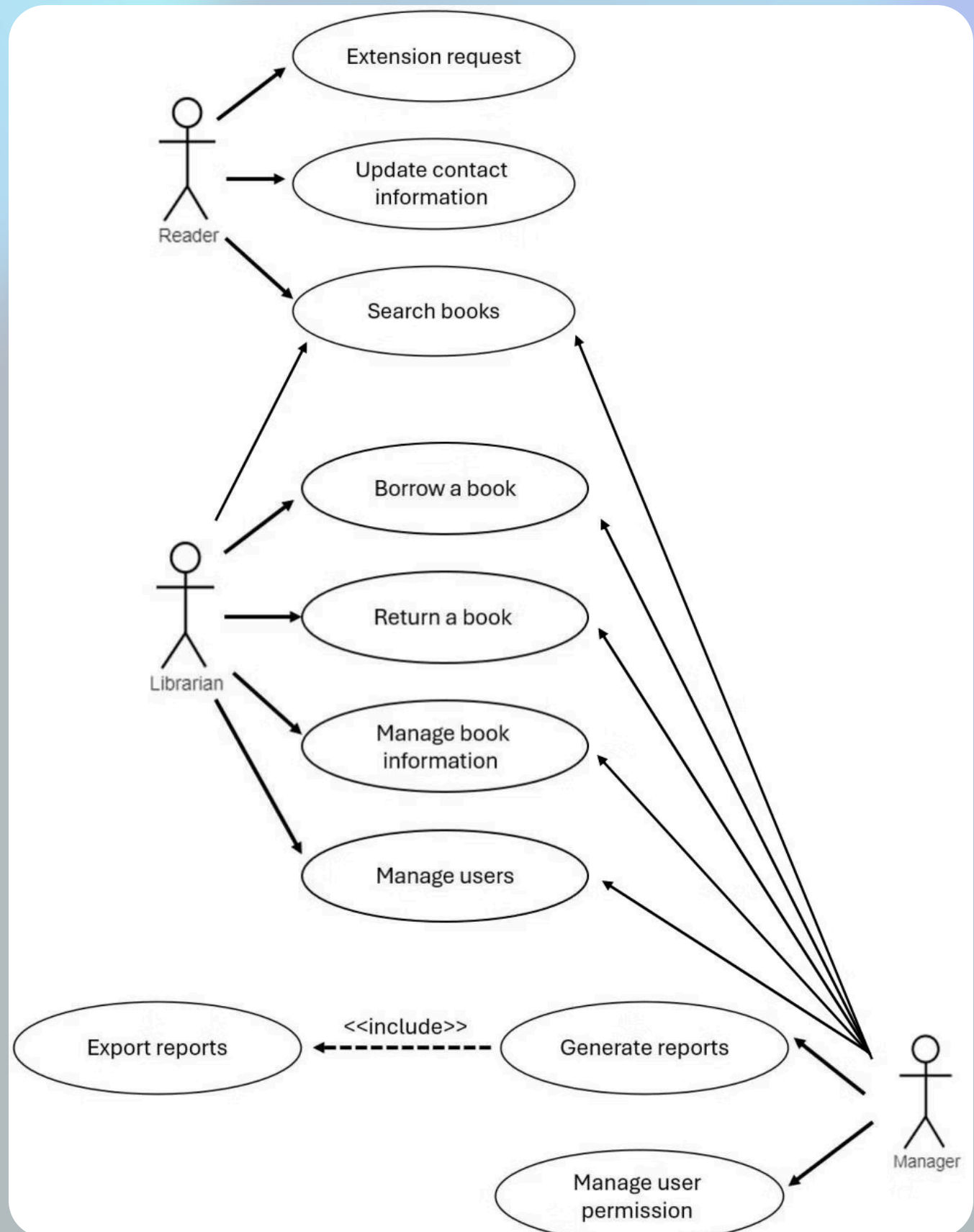
Search, borrow/return books, manage book info and users.

Manager

All librarian capabilities + generate/export reports and manage permissions.

System

Automatic payment, alert before termination, block reader after 3 late returns.



System Actor – Automated Functions

Alert Before Termination

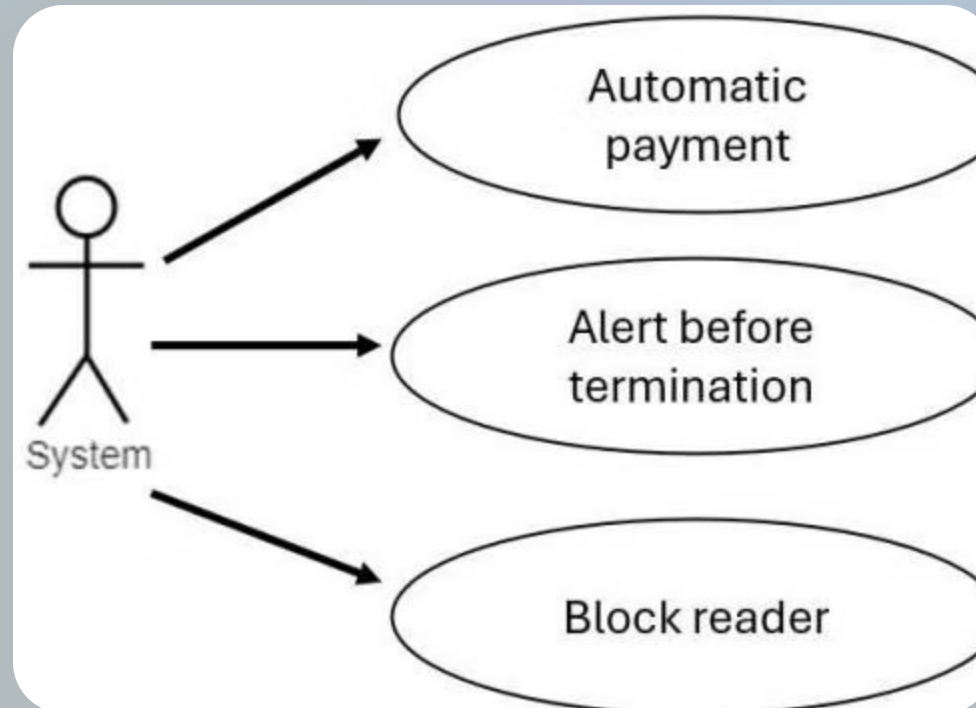
Notifies the reader when the loan period is about to expire – and again before a fine is charged.

Automatic Payment

Automatically charges the reader if the book is not returned on time.

Block Reader

Blocks a reader who has returned books late more than 3 times.



Sequence: Search Book

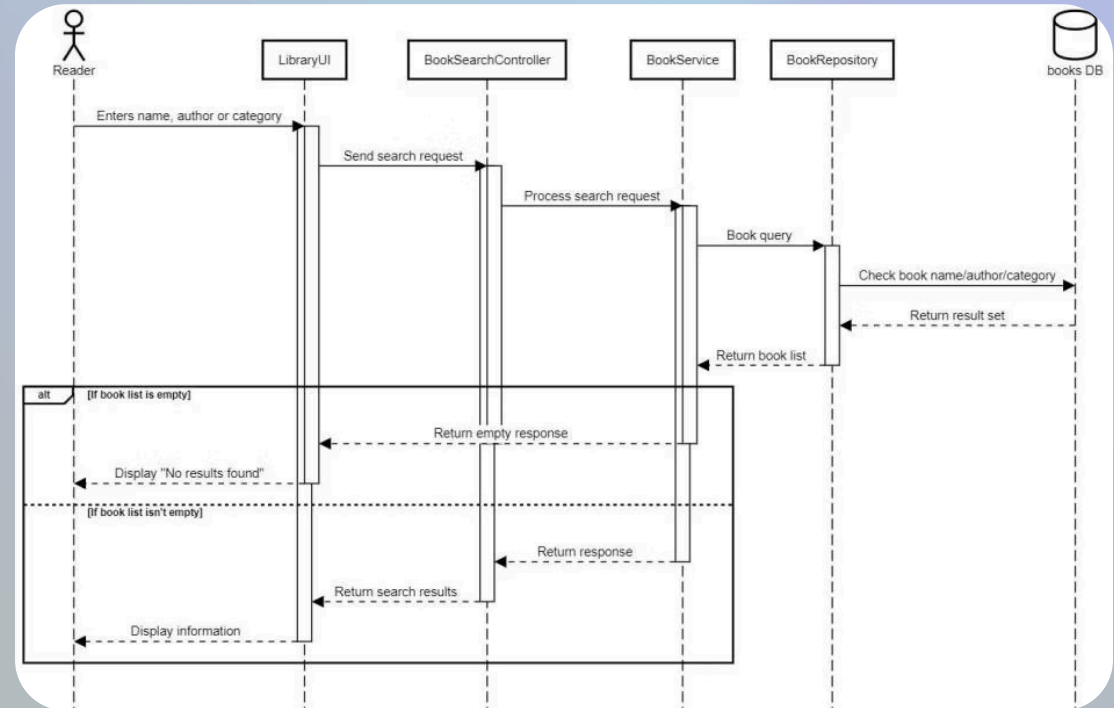
Enter Query

Send to Controller

Service → Repository

DB Returns Results

- ❏ Search supports lookup by title, author, or category.



Sequence: Update Contact Information

01

Reader Input

Reader enters new contact details in LibraryUI.

02

Controller Routes

UserProfileController forwards the update request to Repository.

03

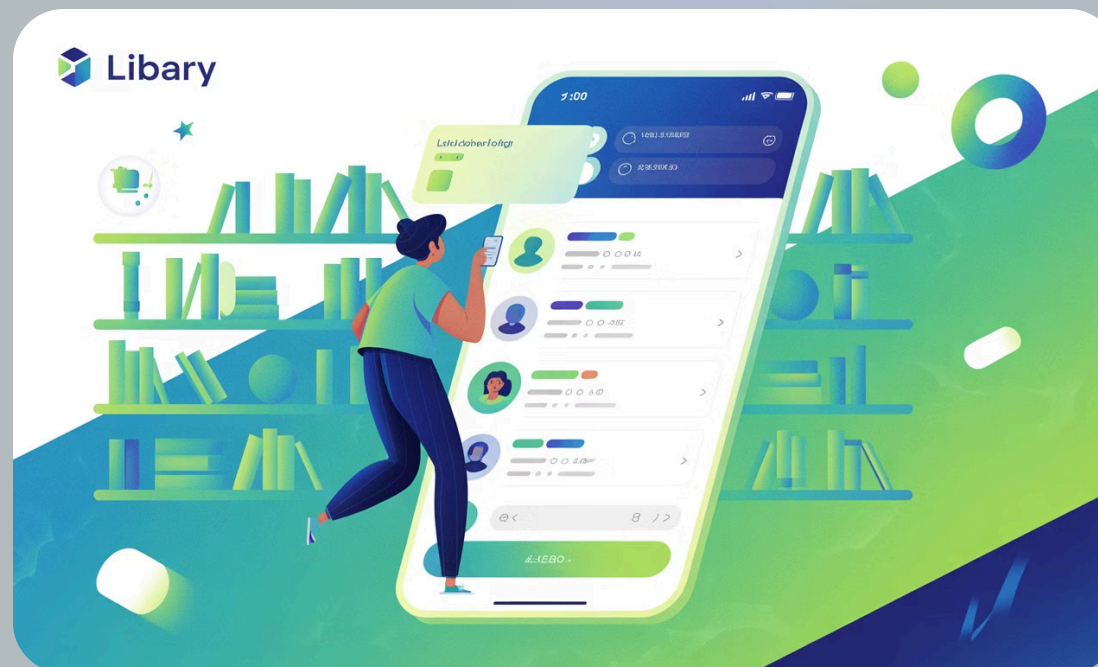
DB Update

Repository executes SQL update query against User DB.

04

Result

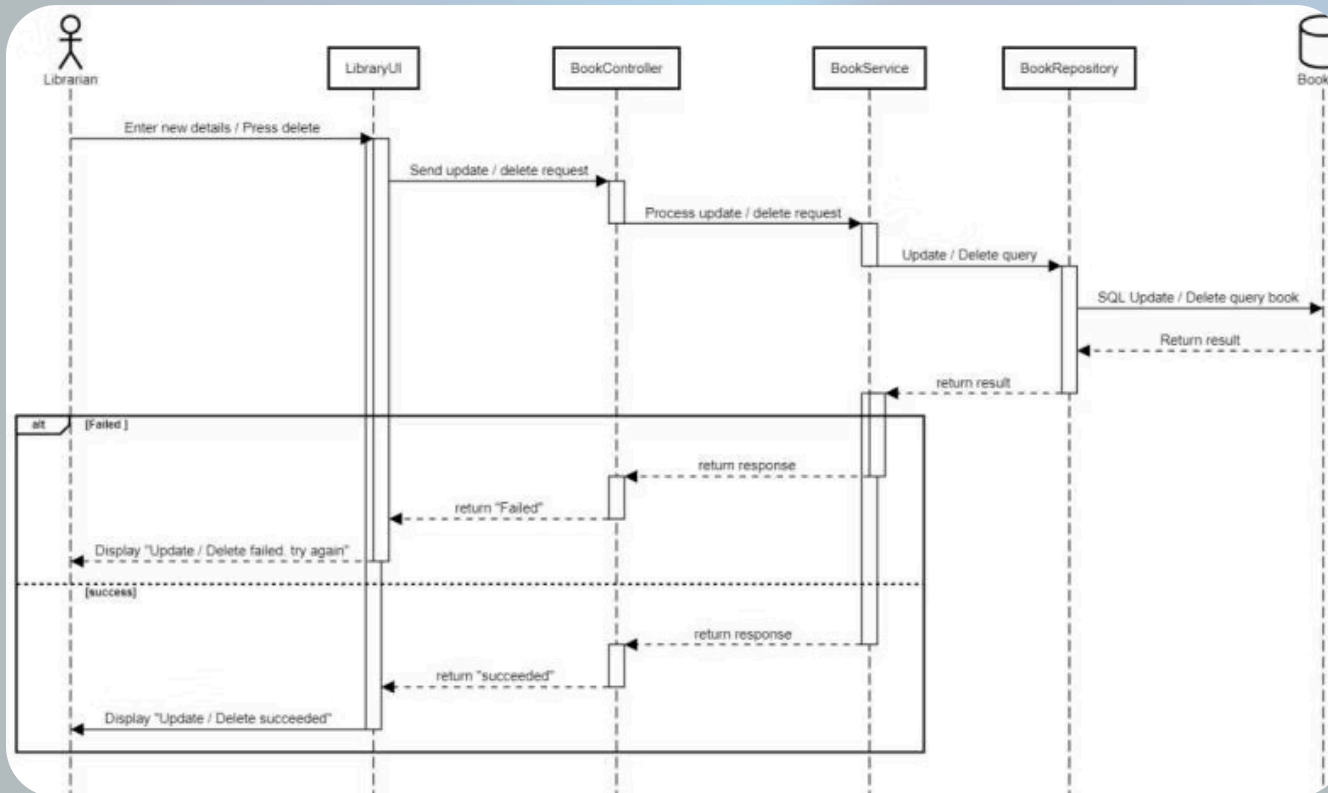
Success: "Update success" shown. **Fail:** "Update failed" shown.



Sequence: Manage Book Information

Actor: Librarian

Enters new book details or requests deletion via LibraryUI.



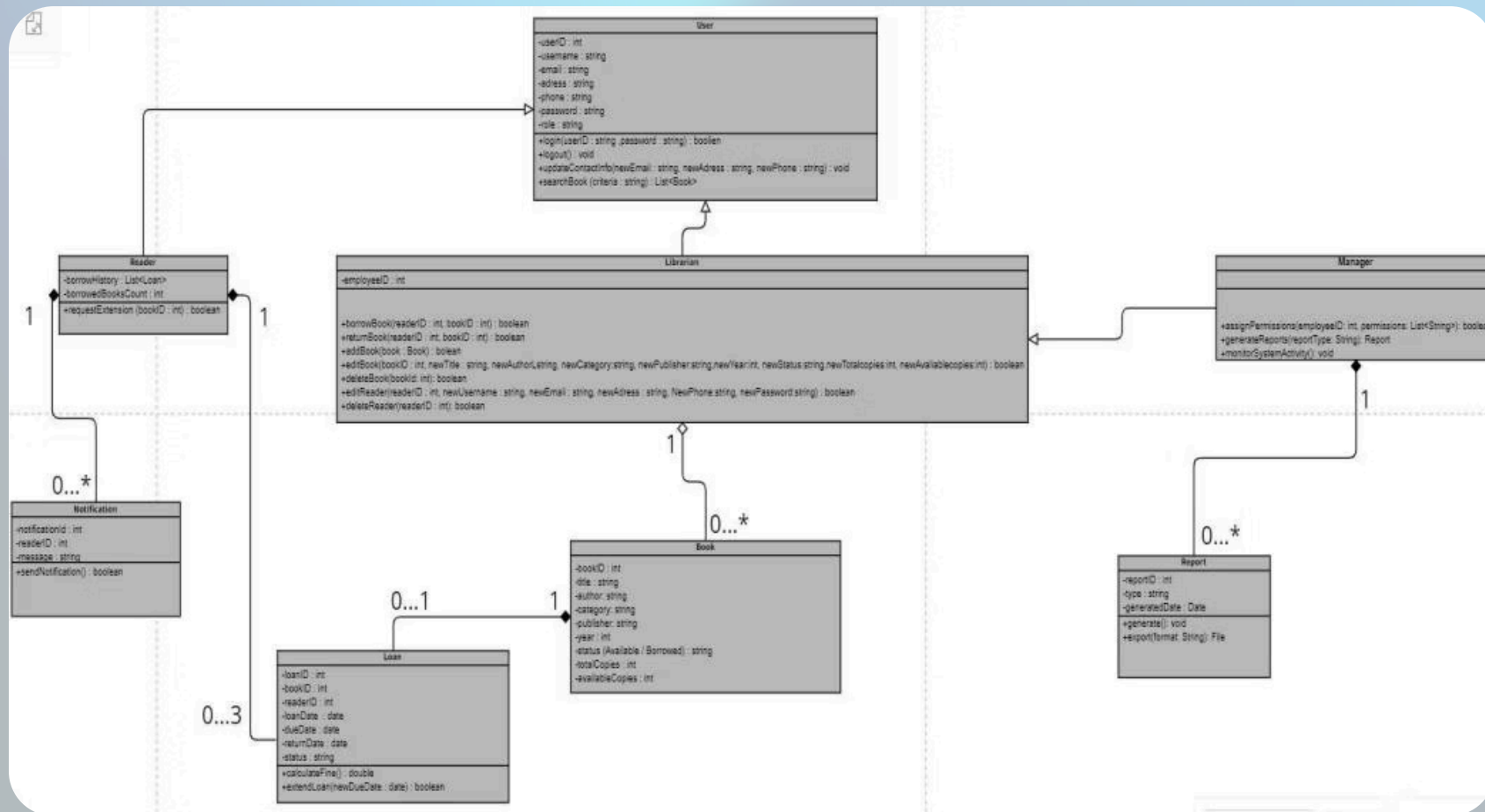
Success

DB confirms → "Update / Delete succeeded" displayed.

Failure

Request failed, book not found, or book currently on loan → "Try again" displayed.

Class Diagram



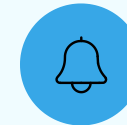
User

Base class. Librarian and Manager inherit from User.



Book

Composed of many Loans (1 to 0..*). Managed by Librarian.



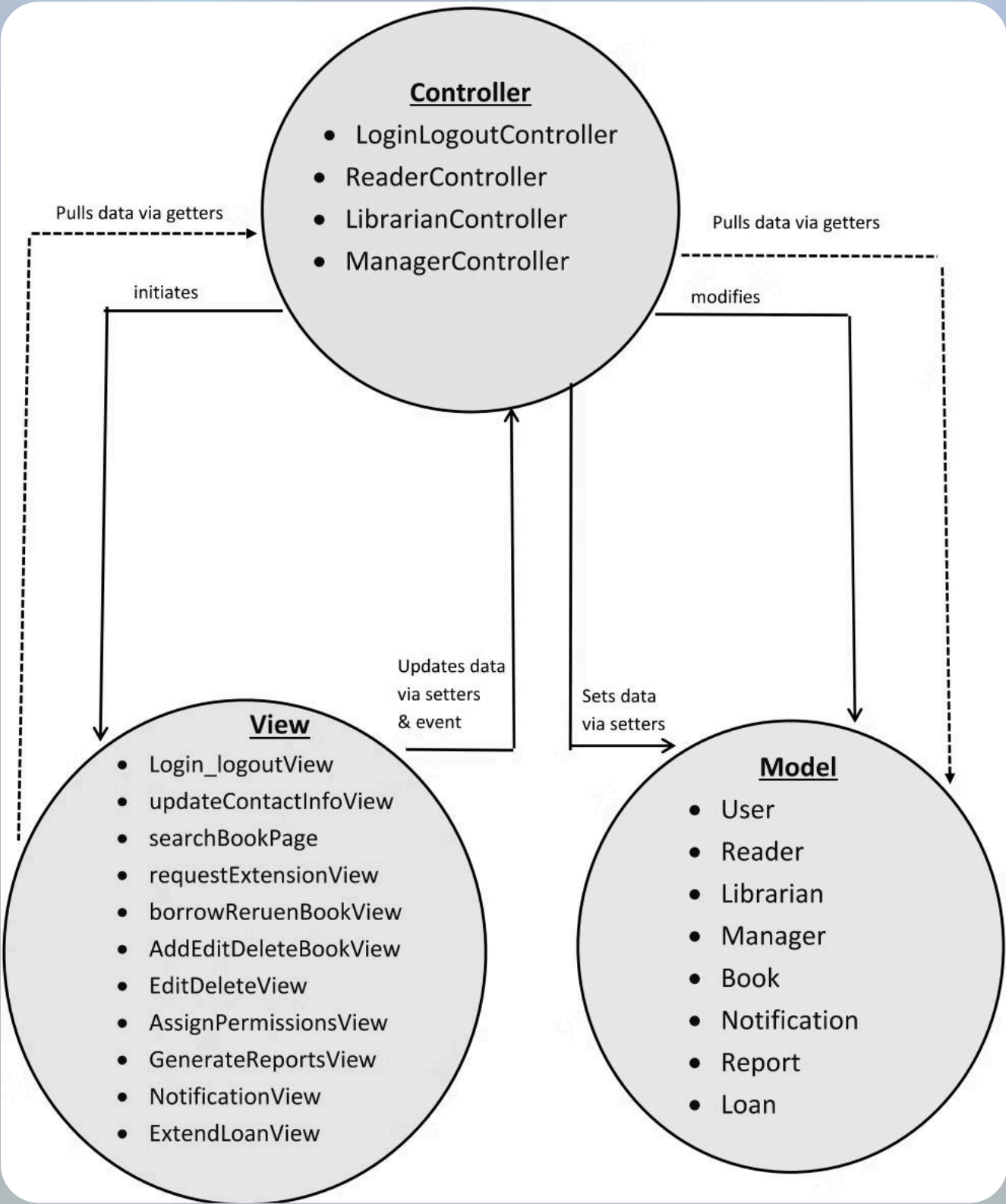
Notification & Loan

Loan tracks borrow/return dates and calculates fines. Notification sends alerts.

MVC Architecture – LibPro

LibPro is designed using the **MVC pattern** – chosen because the system's core function is exposing database data (Model) through a user-friendly interface (View).

→ Model	→ View	→ Controller
User, Reader, Librarian, Manager, Book, Notification, Report, Loan	Login, Search, Borrow/Return, Notifications, Reports, Permissions	LoginLogout, Reader, Librarian, ManagerController – routes requests and updates View





Why MVC?

Separation of Concerns

Data, logic, and UI are cleanly separated for easier maintenance.

Modularity

Each layer can be updated independently without breaking others.

Request Management

Controller ensures proper routing of user requests and internal logic.

Summary



3 Actors + System

Reader, Librarian, Manager — each with defined permissions and use cases.



Sequence Flows

Search, Update Contact, Manage Books — all with success/failure handling.



MVC Architecture

Clean separation of Model, View, and Controller for scalable, maintainable code.

